회로 구성 프로세스 (Circuit Building Process)

개요

Plonky2에서 영지식증명을 생성하기 위해서는 먼저 증명하고자 하는 계산을 산술 회로(Arithmetic Circuit)로 표현해야 합니다. `CircuitBuilder` 는 이러한 회로를 구성하기 위한 핵심 도구로, 개발자가 직관적인 API를 통해 복잡한 계산 로직을 회로로 변환할 수 있게 해줍니다.

이 문서에서는 `CircuitBuilder` 의 주요 메서드들과 회로 구성 프로세스를 단계별로 설명합니다.

CircuitBuilder 구조

`CircuitBuilder` 는 회로 구성에 필요한 모든 상태와 메서드를 포함하는 구조체입니다:

```
pub struct CircuitBuilder<F: RichField + Extendable<D>, const D: usize> {
pub config: CircuitConfig, // 회로 설정
gates: HashSet<GateRef<F, D>>, // 사용된 게이트 타입들
pub(crate) gate_instances: Vec<GateInstance<F, D>>, // 게이트 인스턴스들
public_inputs: Vec<Target>, // 공개 입력
virtual_target_index: usize, // 다음 가상 타겟 인덱스
copy_constraints: Vec<CopyConstraint>, // 복사 제약조건
generators: Vec<WitnessGeneratorRef<F, D>>, // Witness 생성기
constants_to_targets: HashMap<F, Target>, // 상수-타겟 매핑
// ... 기타 필드들
```

```
}
```

1. CircuitBuilder 초기화

1.1 new 함수

회로 구성의 첫 단계는 `CircuitBuilder` 인스턴스를 생성하는 것입니다.

```
pub fn new(config: CircuitConfig) -> Self
```

파라미터:

- `config: CircuitConfig` - 회로의 보안 수준, FRI 설정, 게이트 설정 등을 포함하는 구성 객체

동작 과정:

1. 빈 `CircuitBuilder` 인스턴스 생성
2. `check_config()` 호출하여 설정의 일관성 검증
- FRI 보안 비트가 목표 보안 수준을 만족하는지 확인
- 공식: `fri_security_bits = min(field_bits, num_query_rounds * rate_bits + proof_of_work_bits)`

코드 예시:

```
use plonky2::plonk::circuit_data::CircuitConfig;
use plonky2::plonk::circuit_builder::CircuitBuilder;
use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};

const D: usize = 2;
```

```
type C = PoseidonGoldilocksConfig;

type F = <C as GenericConfig<D>>::F;


// 표준 재귀 설정으로 CircuitBuilder 생성

let  config = CircuitConfig::standard_recursion_config();

let  mut  builder = CircuitBuilder::<F, D>::new(config);
```

1.2 CircuitConfig 주요 설정

`CircuitConfig` 는 회로의 동작을 제어하는 다양한 파라미터를 포함합니다:

- `security_bits` : 목표 보안 수준 (비트 단위)
- `fri_config` : FRI 프로토콜 설정
- `rate_bits` : 코드 레이트
- `cap_height` : Merkle 트리 캡 높이
- `num_query_rounds` : 쿼리 라운드 수
- `proof_of_work_bits` : 작업 증명 비트 수
- `num_routed_wires` : 라우팅 가능한 와이어 수
- `num_constants` : 상수 게이트당 상수 개수
- `use_base_arithmetic_gate` : 기본 산술 게이트 사용 여부
- `max_quotient_degree_factor` : 최대 몫 다항식 차수 계수

2. Target 추가

Target은 회로 내의 변수를 나타내는 추상화된 객체입니다. 두 가지 주요 타입이 있습니다:

- **VirtualTarget**: 실제 와이어가 아닌 가상 타겟으로, witness 생성을 용이하게 함
- **Wire Target**: 특정 게이트의 특정 와이어를 가리키는 구체적인 타겟

2.1 `add_virtual_target` - 단일 가상 타겟 추가

```
pub fn add_virtual_target(&mut self) -> Target
```

가상 타겟을 하나 추가합니다. 가상 타겟은 최종 witness 생성 시 실제 와이어 값으로 변환됩니다.

반환값: 새로 생성된 `Target`

코드 예시:

```
// 두 개의 입력 변수를 위한 가상 타겟 생성
let a = builder.add_virtual_target();
let b = builder.add_virtual_target();
```

2.2 `add_virtual_targets` - 다중 가상 타겟 추가

```
pub fn add_virtual_targets(&mut self, n: usize) -> Vec<Target>
```

n개의 가상 타겟을 벡터로 추가합니다.

코드 예시:

```
// 10개의 타겟을 한 번에 생성
let targets = builder.add_virtual_targets(10);
```

2.3 `constant` - 상수 타겟 생성

```rust
pub fn constant(&mut self, c: F) -> Target
```

주어진 상수 값을 가진 라우팅 가능한 타겟을 반환합니다. 동일한 상수에 대해서는 기존 타겟을 재사용합니다.

동작 과정:

1. `constants_to_targets` 해시맵에서 상수 검색
2. 이미 존재하면 기존 타겟 반환
3. 없으면 새 가상 타겟 생성 후 매핑 저장

코드 예시:

```rust
use plonky2::field::types::Field;


// 상수 0과 1을 가진 타겟 생성
let zero = builder.constant(F::ZERO);
let one = builder.constant(F::ONE);
let five = builder.constant(F::from_canonical_u64(5));
```

2.4 특수 타겟 생성 메서드

```rust
pub fn zero(&mut self) -> Target // 0 값을 가진 타겟
pub fn one(&mut self) -> Target // 1 값을 가진 타겟
pub fn two(&mut self) -> Target // 2 값을 가진 타겟
pub fn neg_one(&mut self) -> Target // -1 값을 가진 타겟
```

2.5 `add_virtual_hash` - 해시 출력 타겟

```
pub fn add_virtual_hash(&mut self) -> HashOutTarget
```

해시 함수의 출력을 나타내는 4개의 타겟으로 구성된 `HashOutTarget` 을 생성합니다.

코드 예시:

```
let hash_output = builder.add_virtual_hash();

// hash_output.elements[0..4]로 각 요소 접근 가능
```

3. 산술 연산 메서드

CircuitBuilder는 다양한 산술 연산을 회로에 추가하는 메서드를 제공합니다. 이러한 연산들은 내부적으로 `ArithmeticGate` 를 사용하여 구현됩니다.

3.1 기본 연산의 핵심: `arithmetic` 메서드

모든 산술 연산의 기반이 되는 메서드입니다:

```
pub fn arithmetic(

&mut self,

const_0: F,

const_1: F,

multiplicand_0: Target,

multiplicand_1: Target,

addend: Target,
```

```
) -> Target
```

계산식: `const_0 * multiplicand_0 * multiplicand_1 + const_1 * addend`

최적화:

1. 특수 케이스 검사 (상수 폴딩, 항등 연산 등)
2. 메모이제이션 - 동일한 연산은 재사용
3. 필요한 경우에만 `ArithmeticGate` 추가

3.2 `add` - 덧셈

```
pub fn add(&mut self, x: Target, y: Target) -> Target
```

구현: `x + y = 1 * x * 1 + 1 * y`

내부적으로 `arithmetic(F::ONE, F::ONE, x, one, y)` 를 호출합니다.

코드 예시:

```
let a = builder.add_virtual_target();
let b = builder.add_virtual_target();
let sum = builder.add(a, b); // sum = a + b
```

3.3 `sub` - 뺄셈

```
pub fn sub(&mut self, x: Target, y: Target) -> Target
```

구현: `x - y = 1 * x * 1 + (-1) * y`

코드 예시:

```
let diff = builder.sub(a, b); // diff = a - b
```

3.4 `mul` - 곱셈

```
pub fn mul(&mut self, x: Target, y: Target) -> Target
```

구현: `x * y = 1 * x * y + 0 * x`

코드 예시:

```
let product = builder.mul(a, b); // product = a * b
```

3.5 `square` - 제곱

```
pub fn square(&mut self, x: Target) -> Target
```

구현: `x^2 = x * x`

코드 예시:

```
let x_squared = builder.square(x); // x_squared = x^2
```

3.6 `mul_add` - 곱셈 후 덧셈

```
pub fn mul_add(&mut self, x: Target, y: Target, z: Target) -> Target
```

구현: `x * y + z`

이 연산은 단일 `ArithmeticGate` 로 효율적으로 처리됩니다.

코드 예시:

```
let result = builder.mul_add(a, b, c); // result = a * b + c
```

3.7 `add_many` - 다중 덧셈

```
pub fn add_many<T>(&mut self, terms: impl IntoIterator<Item = T>) -> Target
where
T: Borrow<Target>
```

여러 타겟을 모두 더합니다.

코드 예시:

```
let terms = vec![a, b, c, d];
let sum = builder.add_many(terms); // sum = a + b + c + d
```

3.8 `mul_many` - 다중 곱셈

```
pub fn mul_many<T>(&mut self, terms: impl IntoIterator<Item = T>) -> Target
where
T: Borrow<Target>
```

코드 예시:

```
let terms = vec![a, b, c];

let product = builder.mul_many(terms); // product = a * b * c
```

3.9 상수 연산

```
pub fn add_const(&mut self, x: Target, c: F) -> Target // x + C

pub fn mul_const(&mut self, c: F, x: Target) -> Target // C * x

pub fn mul_const_add(&mut self, c: F, x: Target, y: Target) -> Target // C *
```

코드 예시:

```
let x_plus_5 = builder.add_const(x, F::from_canonical_u64(5));

let double_x = builder.mul_const(F::TWO, x);
```

3.10 고급 연산

```
pub fn div(&mut self, x: Target, y: Target) -> Target // x / y

pub fn inverse(&mut self, x: Target) -> Target // 1 / x

pub fn neg(&mut self, x: Target) -> Target // -x

pub fn cube(&mut self, x: Target) -> Target // x^3
```

주의: `div` 와 `inverse` 는 y = 0일 때 만족 불가능한 인스턴스를 생성합니다.

4. 제약조건 설정

제약조건은 회로가 만족해야 하는 조건을 정의합니다. Plonky2는 PLONK의 permutation argument를 사용하여 제약조건을 구현합니다.

4.1 `connect` - 두 타겟 연결

```
pub fn connect(&mut self, x: Target, y: Target)
```

두 타겟이 동일한 값을 가져야 한다는 제약조건을 추가합니다. 이는 PLONK의 copy constraint를 통해 구현됩니다.

요구사항: 두 타겟 모두 라우팅 가능해야 합니다.

동작 과정:

1. 두 타겟이 라우팅 가능한지 검증
2. `CopyConstraint` 를 `copy_constraints` 벡터에 추가
3. 빌드 시 permutation argument로 변환

코드 예시:

```
let a = builder.add_virtual_target();
let b = builder.add_virtual_target();
let sum = builder.add(a, b);

// sum이 특정 값과 같아야 한다고 제약
let expected = builder.constant(F::from_canonical_u64(10));
builder.connect(sum, expected);
```

4.2 `assert_zero` - 0 제약

```
pub fn assert_zero(&mut self, x: Target)
```

타겟의 값이 0이어야 한다는 제약조건을 추가합니다.

구현: 내부적으로 `connect(x, zero())` 를 호출

코드 예시:

```
// a - b가 0이어야 함 (즉, a == b)
let diff = builder.sub(a, b);
builder.assert_zero(diff);
```

4.3 `assert_one` - 1 제약

```
pub fn assert_one(&mut self, x: Target)
```

타겟의 값이 1이어야 한다는 제약조건을 추가합니다.

구현: 내부적으로 `connect(x, one())` 를 호출

사용 예시 - 역원 검증:

```
// a가 b의 역원임을 증명
let product = builder.mul(a, b);
builder.assert_one(product); // a * b = 1
```

4.4 조건부 제약조건

```
pub fn conditional_assert_eq(&mut self, condition: Target, x: Target, y: Targ
```

condition 이 참일 때만 `x == y` 를 강제합니다.

구현:

```
// condition이 1이면 x == y, 0이면 제약 없음
let diff = self.sub(x, y);
let constr = self.mul(condition, diff);
self.connect(constr, zero); // condition * (x - y) = 0
```

코드 예시:

```
let is_valid = builder.add_virtual_bool_target_safe();
builder.conditional_assert_eq(is_valid.target, a, b);
// is_valid가 true면 a == b, false면 제약 없음
```

5. 공개 입력 등록

공개 입력(Public Input)은 증명자와 검증자 모두가 알고 있는 값입니다.

5.1 `register_public_input` - 단일 공개 입력

```
pub fn register_public_input(&mut self, target: Target)
```

타겟을 공개 입력으로 등록합니다. 등록된 순서대로 증명의 공개 입력 배열에 포함됩니다.

코드 예시:

```
let result = builder.add(a, b);

builder.register_public_input(result);

// 증명 생성 후 proof.public_inputs[0]으로 접근 가능
```

5.2 `register_public_inputs` - 다중 공개 입력

```
pub fn register_public_inputs(&mut self, targets: &[Target])
```

여러 타겟을 한 번에 공개 입력으로 등록합니다.

코드 예시:

```
let outputs = vec![result1, result2, result3];

builder.register_public_inputs(&outputs);
```

5.3 공개 입력의 역할

빌드 과정에서 공개 입력들은 다음과 같이 처리됩니다:

1. 모든 공개 입력을 해시하여 `public_inputs_hash` 생성
2. `PublicInputGate` 를 추가하여 해시 값 검증
3. 검증자는 이 해시를 사용하여 공개 입력의 무결성 확인

6. 회로 빌드

회로 구성이 완료되면 `build` 메서드를 호출하여 최종 회로 데이터를 생성합니다.

6.1 `build` 메서드

```
pub fn build<C: GenericConfig<D, F = F>>(self) -> CircuitData<F, C, D>
```

`CircuitBuilder` 를 소비하고 증명 생성과 검증에 필요한 모든 데이터를 포함하는 `CircuitData` 를 반환합니다.

반환값: `CircuitData<F, C, D>` - 다음을 포함:

- `prover_only` : 증명자만 필요한 데이터
- `verifier_only` : 검증자만 필요한 데이터
- `common` : 공통 데이터

6.2 빌드 과정 상세

`build` 메서드는 내부적으로 다음 단계를 수행합니다:

6.2.1 공개 입력 처리

```
// 공개 입력 해시 계산
let public_inputs_hash =
self.hash_n_to_hash_no_pad::<C::InnerHasher>(self.public_inputs.clone());


// PublicInputGate 추가 및 연결
let pi_gate = self.add_gate(PublicInputGate, vec![]);
for (&hash_part, wire) in public_inputs_hash.elements.iter()
.zip(PublicInputGate::wires_public_inputs_hash())
{
```

```
self.connect(hash_part, Target::wire(pi_gate, wire))
}
```

6.2.2 상수 생성기 설정

```
// 각 상수-타겟 쌍에 대해 ConstantGenerator 설정
for ((c, t), mut const_gen) in self.constants_to_targets.clone().into_iter()
.sorted_by_key(|(c, _t)| c.to_canonical_u64())
.zip(self.constant_generators.clone())
{
// 상수 다항식에 상수 설정
self.gate_instances[const_gen.row].constants[const_gen.constant_index] = c;
// 타겟과 라우팅 가능한 와이어 연결
self.connect(Target::wire(const_gen.row, const_gen.wire_index), t);
const_gen.set_constant(c);
self.add_simple_generator(const_gen);
}
```

6.2.3 블라인딩 및 패딩

```
self.blind_and_pad();
let degree = self.gate_instances.len();
let degree_bits = log2_strict(degree);
```

회로의 크기를 2의 거듭제곱으로 맞추고, 영지식 속성을 위한 블라인딩 게이트를 추가합니다.

6.2.4 Selector 다항식 생성

```
let (mut constant_vecs, selectors_info) =
selector_polynomials(&gates, &self.gate_instances, quotient_degree_factor + 1);
```

각 게이트 타입에 대한 selector 다항식을 생성합니다. Selector는 특정 행에서 어떤 게이트가 활성화되는지 나타냅니다.

6.2.5 Sigma 다항식 생성

```
let (sigma_vecs, forest) = self.sigma_vecs(&k_is, &subgroup);
```

Copy constraint를 구현하기 위한 permutation 다항식을 생성합니다.

6.2.6 다항식 커밋먼트

```
let constants_sigmas_commitment = PolynomialBatch::<F, C, D>::from_values(
constants_sigmas_vecs,
rate_bits,
PlonkOracle::CONSTANTS_SIGMAS.blinding,
cap_height,
&mut timing,
Some(&fft_root_table),
);
```

상수와 sigma 다항식에 대한 Merkle 트리 기반 커밋먼트를 생성합니다.

6.2.7 최종 데이터 구성

```
CircuitData {
prover_only: ProverOnlyCircuitData { ... },
verifier_only: VerifierOnlyCircuitData { ... },
common: CommonCircuitData { ... },
}
```

7. 완전한 예제: Fibonacci 회로

Fibonacci 수열의 100번째 항을 계산하는 회로를 구성하는 완전한 예제입니다:

```rust
use anyhow::Result;
use plonky2::field::types::Field;
use plonky2::iop::witness::{PartialWitness, WitnessWrite};
use plonky2::plonk::circuit_builder::CircuitBuilder;
use plonky2::plonk::circuit_data::CircuitConfig;
use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};

fn main() -> Result<()> {
// 1. 타입 및 설정 정의
const D: usize = 2;
type C = PoseidonGoldilocksConfig;
type F = <C as GenericConfig<D>>::F;
```

```
// 2. CircuitBuilder 초기화
let config = CircuitConfig::standard_recursion_config();
let mut builder = CircuitBuilder::<F, D>::new(config);

// 3. 초기값을 위한 가상 타겟 추가
let initial_a = builder.add_virtual_target();
let initial_b = builder.add_virtual_target();

// 4. Fibonacci 계산 회로 구성
let mut prev_target = initial_a;
let mut cur_target = initial_b;
for _ in 0..99 {
// 이전 두 항을 더함
let temp = builder.add(prev_target, cur_target);
// 다음 반복을 위해 값 이동
prev_target = cur_target;
cur_target = temp;
}

// 5. 공개 입력 등록
// 초기값 두 개와 결과값을 공개
builder.register_public_input(initial_a);
```

```
builder.register_public_input(initial_b);
builder.register_public_input(cur_target);

// 6. 회로 빌드
let data = builder.build::<C>();

// 7. Witness 설정
let mut pw = PartialWitness::new();
pw.set_target(initial_a, F::ZERO)?; // F(0) = 0
pw.set_target(initial_b, F::ONE)?; // F(1) = 1

// 8. 증명 생성
let proof = data.prove(pw)?;

// 9. 결과 출력
println!(
"100th Fibonacci number mod |F| (starting with {}, {}) is: {}",
proof.public_inputs[0],
proof.public_inputs[1],
proof.public_inputs[2]
);

// 10. 증명 검증
```

```
data.verify(proof)?;

Ok(())

}
```

예제 분석

이 예제에서 생성되는 회로의 특징:

1. **게이트 수**: 약 99개의 `ArithmeticGate` (각 덧셈마다 하나)
2. **공개 입력**: 3개 (initial_a, initial_b, result)
3. **제약조건**:

- 99개의 덧셈 제약: `temp = prev + cur`
- 공개 입력 해시 검증 제약

4. **와이어 연결**: 각 반복에서 이전 결과가 다음 입력으로 연결됨

8. 주요 개념 정리

8.1 Target의 종류

- **VirtualTarget**: 회로 구성 단계에서 사용되는 추상적인 변수
- **Wire Target**: 특정 게이트의 특정 와이어를 가리키는 구체적인 위치

8.2 라우팅 (Routing)

라우팅 가능한 타겟은 copy constraint를 통해 다른 타겟과 연결될 수 있습니다. 모든 VirtualTarget과 일부 Wire Target이 라우팅 가능합니다.

8.3 Copy Constraint vs Arithmetic Constraint

- **Copy Constraint**: `connect` 메서드로 생성, permutation argument로 구현
- **Arithmetic Constraint**: 산술 게이트로 구현, 실제 계산 수행

8.4 메모이제이션

`CircuitBuilder` 는 동일한 연산을 여러 번 수행할 때 결과를 재사용합니다:

- `base_arithmetic_results` : 기본 산술 연산 결과 캐시
- `constants_to_targets` : 상수 타겟 재사용

9. 성능 고려사항

9.1 게이트 최소화

- 가능한 한 상수 폴딩 활용
- `mul_add` 같은 복합 연산 사용
- 동일한 연산 반복 시 자동 메모이제이션 활용

9.2 공개 입력 최소화

공개 입력이 많을수록 증명 크기와 검증 시간이 증가합니다. 필요한 값만 공개하세요.

9.3 회로 크기

최종 회로 크기는 2의 거듭제곱으로 패딩됩니다. 게이트 수가 2^n을 약간 초과하면 2^(n+1)로 패딩되어 비효율적일 수 있습니다.

다음 단계

회로 구성이 완료되면 다음 단계로 진행합니다:

1. **Witness 생성**: 03-proof-generation.md에서 설명

2. **증명 생성**: Prover가 회로와 witness를 사용하여 증명 생성

3. **증명 검증**: Verifier가 증명의 유효성 검증

참고 자료

- 01-architecture.md - Plonky2 전체 아키텍처
- 07-example-fibonacci.md - Fibonacci 예제 상세 분석
- plonky2 소스: `plonky2/src/plonk/circuit_builder.rs`
- plonky2 소스: `plonky2/src/gadgets/arithmetic.rs`

문서 네비게이션

- **이전**: 01. 아키텍처 개요 - plonky2 전체 구조
- **다음**: 03. 증명 생성 프로세스 - Witness 생성 및 증명 생성
- **관련 문서**:
- 07. Fibonacci 예제 - 기본 회로 구성 예제
- 08. Range Check 예제 - Lookup 사용 예제
- 09. Square Root 예제 - Custom Generator 예제
- 10. 재귀 증명 - 고급 회로 구성